FULL-STACK DEVELOPMENT LABORATORY RECORD

A Laboratory Record submitted in fulfillment of the requirements for the course

Full-Stack Development Laboratory

BACHELOR OF TECHNOLOGY

IN

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING(AI&ML)

Submitted by

Pasumarthy Chaitrika Kalyani

(A24126552164)

Under the guidance of

Mrs. Ch. Swathi

Assistant Professor



ANITS

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING(AI&ML)

ANIL NEERUKONDA INSTITUTE OF TECHNOLOGY AND SCIENCES

(UGC AUTONOMOUS)

(Permanently Affiliated to AU, Approved by AICTE and Accredited by NBA&NAAC with 'A+' Grade)

Sangivalasa, Bheemili Mandal, Visakhapatnam Dist. (A.P)

2026-2027

ANIL NEERUKONDA INSTITUTE OF TECHNOLOGY AND SCIENCES
(UGC AUTONOMOUS)

(Affiliated to AU, Approved by AICTE and Accredited by NAAC)
SANGIVALASA, Bheemili Mandal, VISAKHAPATNAM-531162

ANITS

CERTIFICATE

This is to certify that the laboratory record entitled "FULL-STACK DEVELOPMENT LABORATORY" submitted by Pasumarthy Chaitrika Kalyani (Regd. No. A24126552164) in partial fulfillment of the requirements for the Full-Stack Development Laboratory course in the Department of Computer Science and Engineering (AI&ML) during the year 2026-2027 of Anil Neerukonda Institute of Technology and Sciences (A+), Visakhapatnam is a record of bonafide work carried out under my guidance and supervision.

Mrs. Ch. Swathi **Assistant Professor** **(Faculty In-Charge)**	**Head of the Department** **CSE (AI&ML)** **ANITS**	**External Examiner** **Date: _____**

ACKNOWLEDGEMENT

I express my deep gratitude and sincere respect to our respected Head of the Department, Department of Computer Science and Engineering, and the Management of Anil Neerukonda Institute of Technology and Sciences (ANITS), for providing state-of-the-art laboratory computing infrastructure and continuous encouragement throughout this academic year.

I convey my profound appreciation to my faculty guide and Course Instructor, Mrs. Ch. Swathi, Assistant Professor, whose valuable guidance, insightful reviews, and continuous mentorship were instrumental in the successful implementation of all 10 experiments and the MERN stack application.

I also express my thanks to the laboratory technical staff and my classmates for their constructive technical discussions, cooperation, and support throughout the laboratory curriculum.

Pasumarthy Chaitrika Kalyani
Regd. No: A24126552164
B.Tech CSE (AI&ML)
ANITS, Visakhapatnam

List of Experiments

S.No	Description of the Experiment	Course Outcome
1	Design static webpage using HTML components	CO1
2	Design webpage using HTML5 & CSS3	CO1
3	Write a program using JavaScript (arrays, functions)	CO1
4	Develop dynamic webpage using JavaScript	CO1
5	Implement modules in Node.js	CO2
6	Develop mini application using Express and Node.js	CO2
7	Create functional and class components using React	CO3
8	Implement HTTP methods using JSON Server	CO3
9	Develop CRUD operations using MongoDB (Mongoose and mongosh)	CO4
10	Develop main project using Express, React and MongoDB	CO4

EXPERIMENT 1
DESIGN STATIC WEBPAGE USING HTML COMPONENTS

[Course Outcome: CO1]

AIM:

To design a static webpage using basic HTML components such as headings, paragraphs, lists, links, tables and forms.

THEORY:

HTML (HyperText Markup Language) describes the structure of a webpage using elements (tags). Static webpages display fixed content that does not change with user interaction. Common components are headings (<h1>–<h6>), paragraphs (<p>), lists (<ul>, <ol>), links (<a>), images (<img>), tables (<table>) and forms (<form>) with controls such as <input>, <select> and <textarea>.

IMPORTANT CONCEPTS:

- **Document skeleton:** <!DOCTYPE html> declares the document type. <html> is the root, <head> holds metadata (character set, title) that is not displayed, and <body> holds everything visible.
- **Block vs inline:** block elements (<h1>, <p>, <ul>, <table>, <form>) start on a new line and take the full width; inline elements (<a>, , <input>) sit within a line of text.
- **Lists and tables:** is numbered, <ul> is bulleted, each item is an <li>. In a table, <tr> is a row, <th> a header cell and <td> a data cell. Tables are for tabular data, not page layout.
- **Forms:** <input> offers single-line text, <select> with <option> gives a drop-down list, and <textarea> accepts several lines. <label> ties a caption to its control and name identifies the value on submission.

Element	Purpose	Example
<h1> to <h6>	Headings, h1 is the most important	Title
<a href="">	Hyperlink	Tourism
<select>, <option>	Drop-down list of choices	<option>RK Beach</option>
<textarea>	Multi-line text input	<textarea rows="3"></textarea>
<table>, <tr>, <td>	Tabular data	<td>Beach</td>

PROCEDURE:

1. Open a text editor (VS Code, Notepad++ or similar) and create a new file named exp1.html.
2. Type the document skeleton: <!DOCTYPE html>, <html>, <head> with the title, and <body>.
3. Add the headings, paragraph, ordered list, unordered list with links, table and feedback form, in that order.
4. Save the file and open it in a browser by double-clicking it.
5. Check that every element appears as expected; use View Page Source to confirm the markup.

SOURCE CODE:

File Name: exp1.html

```
<!doctype html>
<html lang="en">
  <head>
    <meta charset="UTF-8" />
    <title>Visakhapatnam City Guide</title>
  </head>
  <body>
    <h1>Visakhapatnam City Guide</h1>
    <h3>Full-Stack Development Laboratory - Experiment 1</h3>
    <p>
      Visakhapatnam is a port city on the east coast of Andhra Pradesh.
This static page
      lists a few places worth visiting.
    </p>
    <h2>Top Places to Visit</h2>
    <ol>
      <li>RK Beach</li>
      <li>Kailasagiri Hill Park</li>
      <li>Submarine Museum (INS Kurusura)</li>
      <li>Borra Caves</li>
    </ol>
    <h2>Helpful Links</h2>
    <ul>
      <li><a href="https://www.aptourism.gov.in">Andhra Pradesh
Tourism</a></li>
      <li><a href="https://www.indianrailways.gov.in">Indian
Railways</a></li>
    </ul>
    <h2>Visiting Details</h2>
    <table border="1" cellpadding="6">
      <tr>
        <th>Place</th>
        <th>Type</th>
        <th>Best Time</th>
      </tr>
      <tr>
        <td>RK Beach</td>
        <td>Beach</td>
        <td>Evening</td>
      </tr>
      <tr>
        <td>Kailasagiri</td>
        <td>Hill park</td>
        <td>Morning</td>
      </tr>
      <tr>
        <td>Borra Caves</td>
        <td>Caves</td>
        <td>Winter</td>
      </tr>
    </table>
```

```
    <h2>Feedback Form</h2>
    <form>
      <label>Your name: <input type="text" name="visitor" /></label><br
/><br />
      <label>
        Favourite place:
        <select name="place">
          <option>RK Beach</option>
          <option>Kailasagiri</option>
          <option>Borra Caves</option>
        </select>
      </label>
      <br /><br />
      <label>
        Comments:<br />
        <textarea name="comments" rows="3" cols="30"></textarea>
      </label>
      <br /><br />
      <button type="submit">Send Feedback</button>
    </form>
  </body>
</html>
```

OUTPUT:

The browser displays a main heading, a subheading, a paragraph about the city, a numbered list of four places, a bulleted list of two clickable links, a bordered table of three places with their type and best time, and a feedback form with a name box, a drop-down for the favourite place, a comments box and a Send Feedback button.

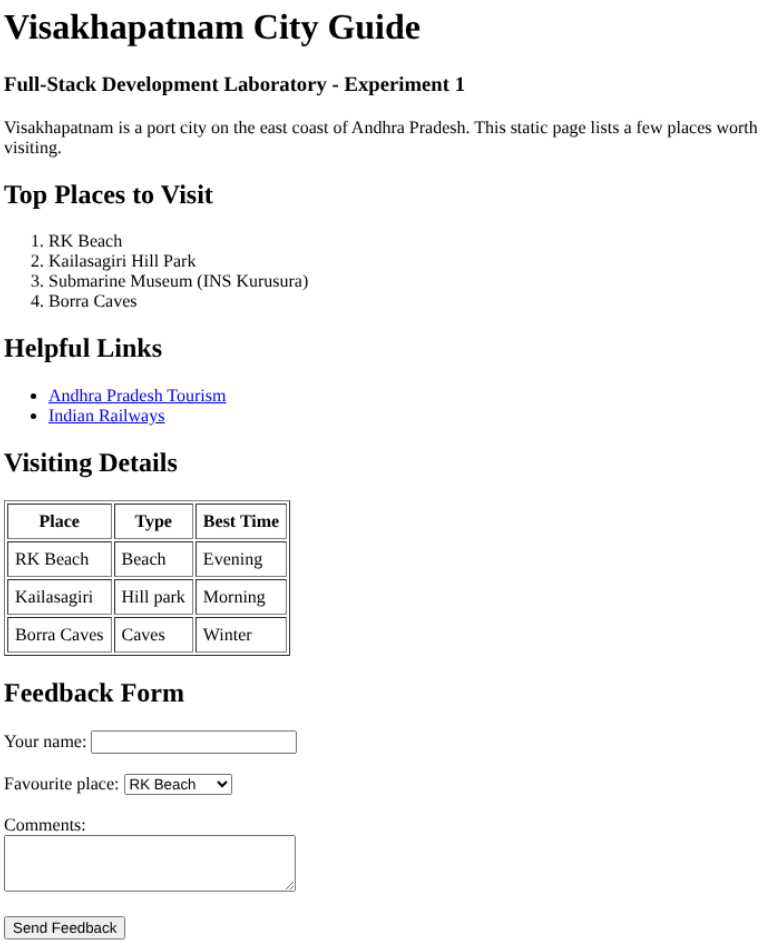


Figure 1: Static webpage rendered in the browser

RESULT:

A static webpage using HTML components was designed and verified in the browser.

VIVA QUESTIONS:

Q1. What is the difference between a block-level and an inline element? Give one example of each.

Ans: A block element starts on a new line and spans the full width (e.g. <p>); an inline element stays within the text flow (e.g. <a>).

Q2. What is the difference between <ol> and ?

Ans: <ol> shows its items with numbers and the order matters; shows bullets and the order does not matter. Both use <li> for each item.

Q3. Which element is used to create a drop-down list, and which element holds each choice?

Ans: <select> creates the drop-down and each choice is an <option> inside it.

Q4. Why should tables not be used for page layout?

Ans: Tables describe tabular data; using them for layout hurts accessibility, responsiveness and maintainability. CSS should be used instead.

EXPERIMENT 2

DESIGN WEBPAGE USING HTML5 & CSS3

[Course Outcome: CO1]

AIM:

To design a webpage using HTML5 semantic elements and style it using CSS3.

THEORY:

HTML5 introduced semantic elements such as <header>, <nav>, <section>, <article>, <aside> and <footer>, which describe the meaning of content. CSS3 controls presentation: colours, fonts, layout, spacing and responsiveness. Styles may be inline, in an internal <style> block, or in an external stylesheet. External stylesheets are preferred because they separate content from presentation.

IMPORTANT CONCEPTS:

- **CSS syntax:** a rule has a selector and a declaration block, e.g. header { background-color: #5a3825; }. Selectors can target an element (nav), a class (.card), an id (#story), a descendant (nav a) or a state (nav a:hover).
- **Cascade and specificity:** inline styles beat id selectors, which beat class selectors, which beat element selectors; later rules win when specificity is equal.
- **Box model:** every element is a box of content, padding, border and margin. Padding is inside the border, margin is outside it.
- **Flexbox:** display: flex arranges children in a row; justify-content spreads them along the main axis. The navigation bar uses justify-content: space-evenly.
- **CSS Grid:** display: grid with grid-template-columns: repeat(3, 1fr) divides a container into three equal columns. The menu cards use it, and gap sets the spacing between them.
- **Responsive design:** the viewport meta tag makes the page scale on phones, and @media (max-width: 600px) applies styles only on narrow screens: the cards become one column and the navigation stacks vertically.

CSS property	Effect
color, background-color	Text and background colours
padding, margin	Space inside and outside the box
border-radius, box-shadow	Rounded corners and drop shadows (menu cards)
display: grid, grid-template-columns, gap	Grid layout (menu cards)
display: flex, justify-content	Flexbox layout (navigation bar)

PROCEDURE:

1. Create a folder, for example exp2, and place index.html and style.css inside it.
2. Write the HTML structure with the semantic elements in the order header, nav, main, aside and footer.
3. Write the CSS rules for each element, starting with body and working down.

4. Open index.html in a browser, then resize the window below 600 px to confirm the media query.

5. Hover over a navigation link to check the colour change and underline.

SOURCE CODE:

File Name: index.html

```
<!doctype html>
<html lang="en">
  <head>
    <meta charset="UTF-8" />
    <meta name="viewport" content="width=device-width, initial-
scale=1.0" />
    <title>Bean &amp; Brew Cafe</title>
    <link rel="stylesheet" href="style.css" />
  </head>
  <body>
    <header>
      <h1>Bean &amp; Brew Cafe</h1>
      <p>Fresh coffee, quiet corners</p>
    </header>
    <nav>
      <a href="#menu">Menu</a>
      <a href="#story">Our Story</a>
      <a href="#visit">Visit Us</a>
    </nav>
    <main>
      <section id="menu">
        <h2>Today's Menu</h2>
        <div class="cards">
          <article class="card">
            <h3>Filter Coffee</h3>
            <p>Strong South Indian decoction with frothy milk.</p>
            <span class="price">Rs. 60</span>
          </article>
          <article class="card">
            <h3>Cold Brew</h3>
            <p>Steeped for 12 hours and served over ice.</p>
            <span class="price">Rs. 150</span>
          </article>
          <article class="card">
            <h3>Veg Puff</h3>
            <p>Baked fresh every morning.</p>
            <span class="price">Rs. 40</span>
          </article>
        </div>
      </section>
      <article id="story">
        <h2>Our Story</h2>
        <p>
          Bean &amp; Brew started as a student project and grew into a
small
```

```
          neighbourhood cafe.
        </p>
      </article>
    </main>
    <aside>
      <p>Offer: Show your college ID and get 10% off every Monday.</p>
    </aside>
    <footer id="visit">
      <p>12 Beach Road, Visakhapatnam &middot; Open 8 AM - 10 PM</p>
    </footer>
  </body>
</html>
```

File Name: style.css

```
body {
  font-family: Georgia, serif;
  margin: 0;
  background-color: #fdf6ec;
  color: #3b2a20;
}
header {
  background-color: #5a3825;
  color: #fdf6ec;
  text-align: center;
  padding: 24px;
}
header h1 {
  margin: 0;
}
header p {
  margin: 6px 0 0;
  font-style: italic;
}
nav {
  background-color: #7b4f35;
  display: flex;
  justify-content: space-evenly;
  padding: 12px;
}
nav a {
  color: #fdf6ec;
  text-decoration: none;
  letter-spacing: 1px;
}
nav a:hover {
  color: #ffd27f;
  text-decoration: underline;
}
main {
  padding: 20px;
}
```

```
.cards {
  display: grid;
  grid-template-columns: repeat(3, 1fr);
  gap: 16px;
}
.card {
  background: white;
  padding: 14px;
  border-radius: 10px;
  box-shadow: 0 3px 8px rgba(0, 0, 0, 0.15);
}
.price {
  display: inline-block;
  background: #5a3825;
  color: white;
  padding: 3px 10px;
  border-radius: 12px;
}
#story {
  background: #fff;
  padding: 15px;
  margin-top: 20px;
  border-left: 5px solid #7b4f35;
}
aside {
  background-color: #ffe9bf;
  margin: 0 20px;
  padding: 12px;
  text-align: center;
}
footer {
  background-color: #5a3825;
  color: #fdf6ec;
  text-align: center;
  padding: 12px;
  margin-top: 20px;
}
@media (max-width: 600px) {
  .cards {
    grid-template-columns: 1fr;
  }
  nav {
    flex-direction: column;
    align-items: center;
    gap: 8px;
  }
}
```

OUTPUT:

The page shows a dark brown header with a tagline, a navigation bar with three links (gold and underlined on hover), three white menu cards with rounded corners, shadows and price tags laid out in a grid, a story box with a left border, a light offer box, and a dark footer. On narrow screens the cards stack into a single column and the navigation links stack vertically.

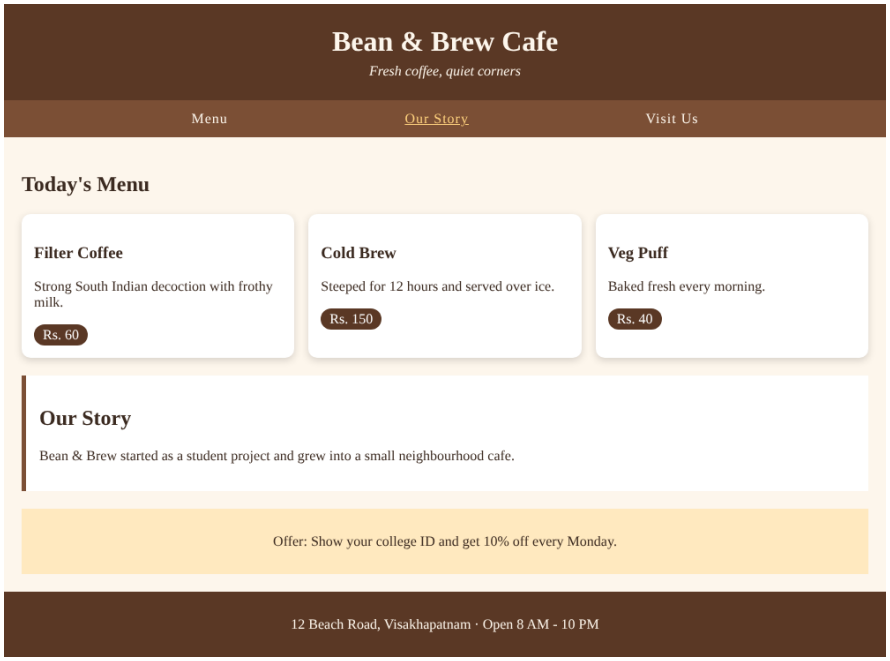


Figure 2(a): Cafe webpage at desktop width, pointer over the Our Story link

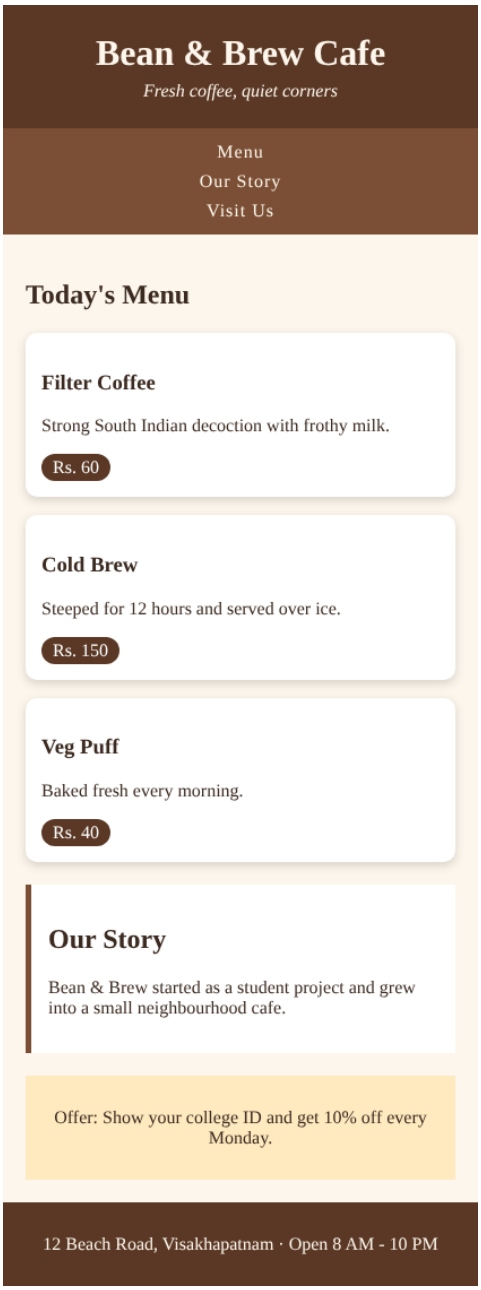


Figure 2(b): Same page at 420 px width; cards and navigation links stacked by the media query

RESULT:

A responsive webpage was designed using HTML5 semantic elements and styled with CSS3.

VIVA QUESTIONS:

Q1. Why are semantic elements preferred over generic elements?

Ans: They convey meaning to browsers, search engines and screen readers, and make the code easier to read.

Q2. What does grid-template-columns: repeat(3, 1fr) do?

Ans: It creates three columns of equal width, where 1fr is one equal share of the free space; the menu cards fill these columns.

Q3. How does specificity decide which CSS rule applies?

Ans: The rule with the more specific selector wins (inline > id > class > element); with equal specificity the later rule wins.

Q4. What does the viewport meta tag do on a mobile device?

Ans: It sets the page width to the device width so the layout scales instead of being shown zoomed out.

EXPERIMENT 3

WRITE A PROGRAM USING JAVASCRIPT

(ARRAYS, FUNCTIONS)

[Course Outcome: CO1]

AIM:

To write JavaScript programs that use arrays and user-defined functions.

THEORY:

An array is an ordered collection of values accessed by index. JavaScript provides methods such as push(), pop(), sort(), map(), filter() and reduce(). A function is a reusable block of code defined with the function keyword or as an arrow function; it may accept parameters and return a value. This experiment analyses a list of student marks with both.

IMPORTANT CONCEPTS:

- **Arrays:** values are stored by index starting at 0; arr.length gives the number of elements.
- **Functions:** declared with function name(params) { ... } or as arrow functions such as m => m + 5. Parameters are local and return sends a value back.
- **Sorting:** sort() converts items to text by default, so numbers need a comparator: (a, b) => b - a sorts from high to low. slice() first makes a copy so the original array is not changed.
- **Recursion:** factorial(n) calls itself with n - 1 until it reaches the base case n <= 1.
- **Palindrome check:** the text is lower-cased, every character that is not a letter or digit is removed with a regular expression, and the result is compared with its reverse.

Method	Purpose	Modifies original?
push() / pop()	Add or remove at the end	Yes
slice()	Copy a portion of the array	No
sort()	Sort the elements	Yes (so copy first)
map()	Transform each element, return a new array	No
filter()	Keep elements that satisfy a condition	No
reduce()	Combine all elements into one value	No

PROCEDURE:

1. Create a file exp3.js in a folder.
2. Define the functions calculateAverage, findMin, sortDescending, getGrade, factorial and isPalindrome.
3. Declare the marks array, call each function and print the results with console.log.
4. Use filter for the passed marks, map with getGrade for the grades and map with an arrow function for the bonus marks, then reduce for the total.
5. Run the file with node exp3.js and compare the output with the values worked out by hand.

RUN:

Commands
node exp3.js

SOURCE CODE:

File Name: exp3.js

```
// Student marks analysis using arrays and functions
function calculateAverage(marks) {
  let total = 0;
  for (let i = 0; i < marks.length; i++) {
    total += marks[i];
  }
  return total / marks.length;
}

function findMin(marks) {
  let min = marks[0];
  for (let i = 1; i < marks.length; i++) {
    if (marks[i] < min) min = marks[i];
  }
  return min;
}

function sortDescending(marks) {
  return marks.slice().sort((a, b) => b - a);
}

function getGrade(mark) {
  if (mark >= 90) return "A+";
  if (mark >= 75) return "A";
  if (mark >= 60) return "B";
  if (mark >= 40) return "C";
  return "F";
}

function factorial(n) {
  if (n <= 1) return 1;
  return n * factorial(n - 1);
}

function isPalindrome(text) {
  const clean = text.toLowerCase().replace(/[^a-z0-9]/g, "");
  return clean === clean.split("").reverse().join("");
}

// Main program
const marks = [78, 92, 45, 66, 88, 35, 71];
console.log("Marks:", marks);
```

```
console.log("Average:", calculateAverage(marks).toFixed(2));
console.log("Lowest mark:", findMin(marks));
console.log("Sorted (high to low):", sortDescending(marks));

const passed = marks.filter((m) => m >= 40);
console.log("Passed marks:", passed);
console.log("Number failed:", marks.length - passed.length);

const grades = marks.map(getGrade);
console.log("Grades:", grades);

const bonus = marks.map((m) => Math.min(m + 5, 100));
console.log("After 5 bonus marks:", bonus);

const total = marks.reduce((sum, m) => sum + m, 0);
console.log("Total using reduce:", total);

console.log("Factorial of 6:", factorial(6));
console.log("Is 'Madam' a palindrome?", isPalindrome("Madam"));
console.log("Is 'Hello' a palindrome?", isPalindrome("Hello"));
```

OUTPUT:

Hand check: the seven marks add up to 475, so the average is 475 / 7 = 67.86. Only 35 is below the pass mark of 40, so six marks pass and one fails. 6! = 720.

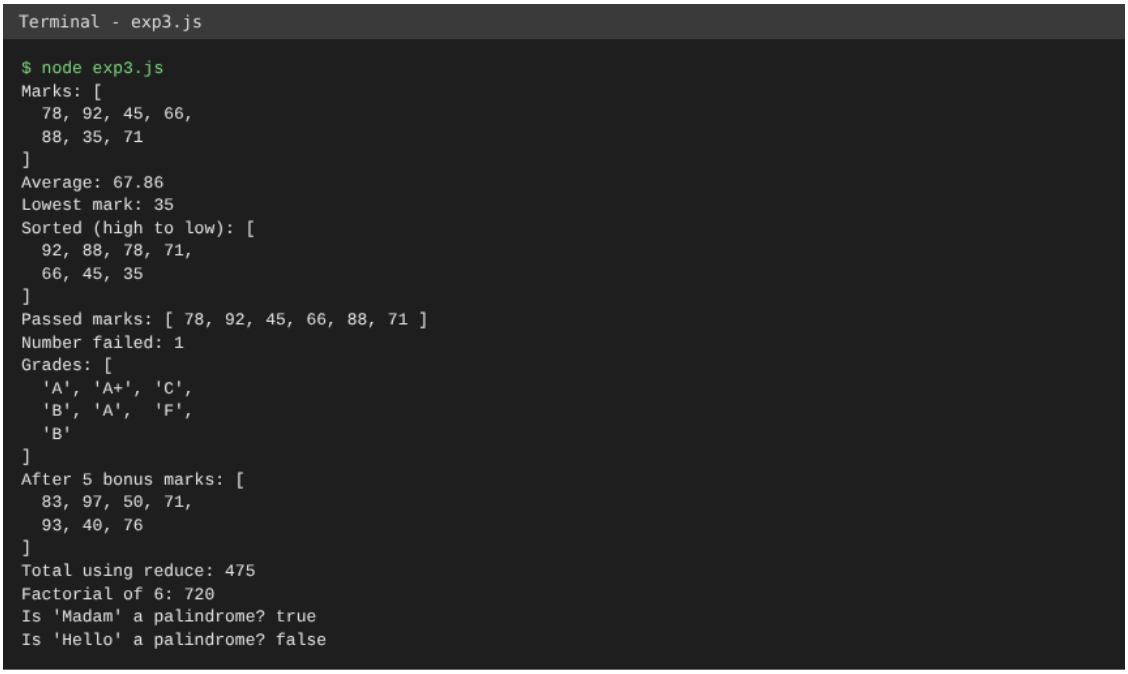


Figure 3: Terminal output of exp3.js

RESULT:

JavaScript programs using arrays and functions were written and executed successfully.

VIVA QUESTIONS:

Q1. What is the difference between map() and filter()?

Ans: map() transforms every element and returns an array of the same length; filter() returns only the elements that pass a test.

Q2. Why is a comparator needed in sort((a, b) => b - a)?

Ans: By default sort() compares items as text, so 100 would come before 9. The comparator compares numbers and b - a gives descending order.

Q3. What is the base case in the factorial function and why is it needed?

Ans: The base case is n <= 1, which returns 1. Without it the function would call itself forever and cause a stack overflow.

Q4. What is the difference between a function declaration and an arrow function?

Ans: Declarations are hoisted; arrow functions are shorter and inherit this from the surrounding scope.

EXPERIMENT 4

DEVELOP DYNAMIC WEBPAGE USING

JAVASCRIPT

[Course Outcome: CO1]

AIM:

To develop a dynamic webpage that responds to user events and modifies the DOM using JavaScript.

THEORY:

The Document Object Model (DOM) represents an HTML document as a tree of objects. JavaScript can access elements with document.getElementById() and querySelector(), change their content or style, create new elements and respond to events such as click, input and mouseover. This makes the page dynamic without reloading. The page built here has three small utilities: a bill splitter, a live character counter and a shopping list, plus a dark-mode switch.

IMPORTANT CONCEPTS:

- **Selecting elements:** getElementById("id") returns one element; querySelector("css") returns the first match; querySelectorAll("css") returns all matches.
- **Events:** handlers can be attached inline (onclick="splitBill()") or with addEventListener, which keeps HTML and JavaScript separate. The input event fires on every keystroke, which drives the live counter.
- **Reading input:** an input's value is always text, so Number(...) converts it before arithmetic; toFixed(2) rounds the answer to two decimals.
- **Toggling a class:** classList.toggle("dark") adds the class if it is missing and removes it if present, so one rule in the CSS restyles the whole page.
- **Creating and removing elements:** createElement("li") creates a node, textContent sets its text and appendChild attaches it; element.remove() deletes it again.
- **Validation:** text.trim() === "" rejects empty or whitespace-only input before an item is added, and the maxlength attribute stops the note at 60 characters.

PROCEDURE:

1. Create exp4.html with a theme button and three fieldsets: bill splitter, character counter and shopping list.
2. Add the CSS in a <style> block, including the body.dark rule and the .limit class for the counter.
3. Write splitBill and addItem in a <script> block, then attach the input listener to the note and the click listener to the theme button.
4. Open the page, split a bill, type in the note box until the counter reaches 60, and switch the theme.
5. Add some items, remove one with its Remove button, and click Add with an empty box to see the alert.

SOURCE CODE:

File Name: exp4.html

```
<!doctype html>
<html lang="en">
  <head>
    <meta charset="UTF-8" />
    <title>Dynamic Webpage - Mini Utilities</title>
    <style>
      body {
        font-family: Arial, sans-serif;
        max-width: 460px;
        margin: 30px auto;
        background: #ffffff;
        color: #222;
      }
      body.dark {
        background: #222;
        color: #f1f1f1;
      }
      fieldset {
        margin-bottom: 18px;
        border: 1px solid #999;
      }
      textarea {
        width: 95%;
      }
      #counter {
        font-size: 14px;
      }
      #counter.limit {
        color: crimson;
        font-weight: bold;
      }
      li {
        margin: 4px 0;
      }
      li button {
        margin-left: 8px;
      }
    </style>
  </head>
  <body>
    <h1>Mini Utilities</h1>
    <button id="themeBtn">Toggle Dark Mode</button>

    <fieldset>
      <legend>Bill Splitter</legend>
      Bill amount:
      <input type="number" id="bill" value="0" min="0" /><br /><br />
      People: <input type="number" id="people" value="1" min="1" /><br
/><br />
```

```
      <button onclick="splitBill()">Split Bill</button>
      <p id="splitResult"></p>
    </fieldset>

    <fieldset>
      <legend>Live Character Counter</legend>
      <textarea
        id="note"
        rows="3"
        maxlength="60"
        placeholder="Type a note (max 60)"
      ></textarea>
      <p id="counter">0 / 60 characters</p>
    </fieldset>

    <fieldset>
      <legend>Shopping List</legend>
      <input type="text" id="item" placeholder="Add an item" />
      <button onclick="addItem()">Add</button>
      <ul id="list"></ul>
    </fieldset>

    <script>
      function splitBill() {
        const bill = Number(document.getElementById("bill").value);
        const people = Number(document.getElementById("people").value);
        const out = document.getElementById("splitResult");
        if (bill <= 0 || people < 1) {
          out.textContent = "Enter a valid bill and number of people.";
          return;
        }
        out.textContent = "Each person pays Rs. " + (bill /
people).toFixed(2);
      }

      const note = document.getElementById("note");
      const counter = document.getElementById("counter");
      note.addEventListener("input", () => {
        counter.textContent = note.value.length + " / 60 characters";
        counter.classList.toggle("limit", note.value.length >= 60);
      });

      document.getElementById("themeBtn").addEventListener("click", ()
=> {
        document.body.classList.toggle("dark");
      });

      function addItem() {
        const input = document.getElementById("item");
        const text = input.value.trim();
        if (text === "") {
          alert("Please type an item first.");
```

```
          return;
        }
        const li = document.createElement("li");
        li.textContent = text;
        const del = document.createElement("button");
        del.textContent = "Remove";
        del.addEventListener("click", () => li.remove());
        li.appendChild(del);
        document.getElementById("list").appendChild(li);
        input.value = "";
      }
    </script>
  </body>
</html>
```

OUTPUT:

On loading, the page shows the Toggle Dark Mode button and the three fieldsets. In the test run the bill of 1250 split between 5 people displayed “Each person pays Rs. 250.00”. Typing a long note stopped at the 60-character limit and the counter turned red. Three items were added (Notebook, Pen, Graph paper) and Pen was removed. Clicking Add with an empty box showed the alert “Please type an item first.”. The theme button switched the page to dark mode.



Figure 4(a): Dynamic webpage on initial load

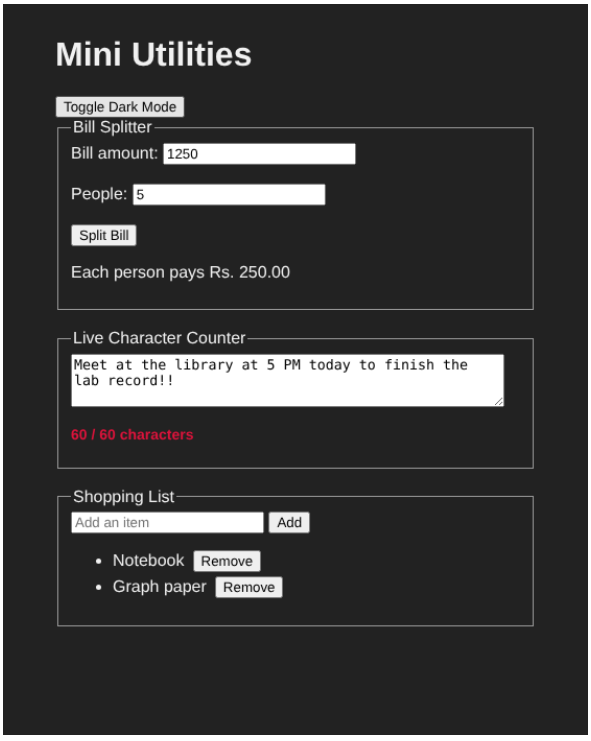


Figure 4(b): After the test run, in dark mode

RESULT:

A dynamic webpage responding to user events was developed using JavaScript and the DOM.

VIVA QUESTIONS:

Q1. What is the DOM?

Ans: The Document Object Model is a tree representation of the page that JavaScript can read and modify.

Q2. Why is Number() used on the value of an input before dividing?

Ans: The value of an input is always a string; Number() converts it to a number so that bill / people is real division.

Q3. Which event is used for the live character counter and why?

Ans: The input event, because it fires on every change to the text (typing, pasting, deleting), unlike change, which fires only when the box loses focus.

Q4. How does classList.toggle help to build the dark mode?

Ans: It adds or removes the dark class on <body> with one call, and the CSS rule body.dark restyles the whole page at once.

EXPERIMENT 5

IMPLEMENT MODULES IN NODE.JS

[Course Outcome: CO2]

AIM:

To create and use custom and built-in modules in Node.js.

THEORY:

Node.js follows the CommonJS module system. Each file is a module. A module exposes functions or variables using module.exports (or the shorthand exports) and another file imports them with require(). Node.js also provides built-in modules such as fs, path, os, events and http, which are imported the same way without installation.

IMPORTANT CONCEPTS:

- **CommonJS:** every file has its own scope; variables are private unless exported. A module is loaded once and then cached.
- **Two export styles:** module.exports = { a, b } replaces the whole export object (converter.js), while exports.name = ... adds one property at a time (textUtils.js).
- **Destructured import:** const { capitalize, countWords } = require("./textUtils") picks only the functions that are needed.
- **Relative and package imports:** require("./converter") loads a local file; require("express") loads a package from node_modules.
- **Events:** an EventEmitter registers a listener with on("order", fn) and calls it with emit("order", data); this is the base of many Node.js APIs.
- **Globals:** __filename is the full path of the current file and __dirname is its folder.

Module	Use in this experiment
fs	mkdirSync, writeFileSync, appendFileSync, readFileSync, readdirSync
path	path.join, path.basename, path.extname
os	os.cpus() to read information about the machine
events	EventEmitter with on() and emit()

PROCEDURE:

1. Create converter.js with three functions and export them with module.exports.

2. Create textUtils.js and export three helpers with exports.name.

3. Create app.js and import both modules, then import the built-in modules fs, path, os and events.

4. Call each function, create the notes folder, write and append to log.txt, read it back and emit an event.

5. Run node app.js and check the output and the new notes/log.txt file in the folder.

RUN:

Commands

```
node app.js
```

SOURCE CODE:

File Name: converter.js

```
// Custom module 1: unit conversions
function celsiusToFahrenheit(c) {
  return (c * 9) / 5 + 32;
}
function kmToMiles(km) {
  return km * 0.621371;
}
function inrToUsd(inr, rate = 83) {
  return inr / rate;
}
module.exports = { celsiusToFahrenheit, kmToMiles, inrToUsd };
```

File Name: textUtils.js

```
// Custom module 2: text helpers
exports.capitalize = (s) => s.charAt(0).toUpperCase() +
s.slice(1).toLowerCase();
exports.countWords = (s) => s.trim().split(/\s+/).length;
exports.reverseWords = (s) => s.split(" ").reverse().join(" ");
```

File Name: app.js

```
// Importing custom modules
const converter = require("./converter");
const { capitalize, countWords, reverseWords } = require("./textUtils");

// Importing built-in modules
const fs = require("fs");
const path = require("path");
const os = require("os");
const EventEmitter = require("events");

console.log(
  "37 C in Fahrenheit:",
  converter.celsiusToFahrenheit(37).toFixed(1),
);
console.log("42 km in miles:", converter.kmToMiles(42).toFixed(2));
console.log("Rs. 8300 in USD:", converter.inrToUsd(8300).toFixed(2));

console.log(capitalize("fULL sTACK"));
console.log("Word count:", countWords("node js modules are easy"));
console.log("Reversed:", reverseWords("one two three"));
```

```
// path module
const file = path.join(__dirname, "notes", "log.txt");
console.log("Joined path:", path.basename(file), "|",
path.extname(file));

// fs module: create folder, write, append, read
fs.mkdirSync(path.join(__dirname, "notes"), { recursive: true });
fs.writeFileSync(file, "Line 1: created with fs\n");
fs.appendFileSync(file, "Line 2: appended later\n");
console.log("File content:\n" + fs.readFileSync(file, "utf8").trim());
console.log(
  "Files in notes folder:",
  fs.readdirSync(path.join(__dirname, "notes")),
);

// os module
console.log("CPU cores available:", os.cpus().length > 0);

// events module
const emitter = new EventEmitter();
emitter.on("order", (item) => console.log("Event received: order for",
item));
emitter.emit("order", "Filter Coffee");
```

OUTPUT:

The output below was produced by running the program. The number of CPU cores differs from machine to machine, so only a true/false check is printed.



Figure 5: Terminal output of app.js

RESULT:

Custom and built-in Node.js modules were created, imported and used successfully.

VIVA QUESTIONS:

Q1. What is the difference between module.exports and exports?

Ans: exports is a shortcut reference to module.exports; reassigning exports breaks the link, while assigning module.exports always works.

Q2. Why does require("./converter") need the ./ prefix?

Ans: Without it Node.js looks for a built-in module or a package in node_modules; ./ marks a local file path.

Q3. Why is path.join used instead of joining strings with "/"?

Ans: path.join uses the correct separator for the operating system (\ on Windows, / on Linux and macOS), so the code works on every machine.

Q4. What is the difference between writeFileSync and appendFileSync?

Ans: writeFileSync creates the file or overwrites its content; appendFileSync adds new text at the end and keeps the existing content.

EXPERIMENT 6

DEVELOP MINI APPLICATION USING EXPRESS AND NODE.JS

[Course Outcome: CO2]

AIM:

To develop a small web application using the Express framework on Node.js.

THEORY:

Express is a minimal and flexible Node.js web application framework. It provides routing, middleware support and easy handling of HTTP requests and responses. Routes map URLs and HTTP methods to handler functions, and middleware such as express.json() parses incoming request bodies. The application built here is a small library API that lists, adds, issues and deletes books.

IMPORTANT CONCEPTS:

- **Routes:** app.get, app.post, app.put and app.delete register handlers for a method and path. Path parameters such as :id are read from req.params.
- **Query strings:** a URL such as /api/books?available=true is read from req.query, which is used here to filter the list.
- **Request and response:** req.body holds parsed JSON (after express.json()); res.json() sends the reply and res.status(201) sets the status code.
- **Middleware:** functions that run before the route handler. The logging middleware prints the method and URL of every request and calls next() to continue; express.json() parses the body.
- **Validation and errors:** the POST route returns 400 when title or author is missing, and GET or PUT return 404 when the id does not exist.
- **Limitation:** books are kept in a variable in memory, so they are lost when the server stops. A database such as MongoDB (Experiment 9) solves this. A nextId counter is used so ids are never reused after a delete.

Code	Meaning	Where used
200	OK	GET, PUT, DELETE
201	Created	POST
400	Bad Request	POST without title or author
404	Not Found	Book id that does not exist

PROCEDURE:

1. Create the project folder, run npm init -y and npm install express.

2. Write server.js with the logging middleware and the routes shown below.

3. Run node server.js and open http://localhost:3000 in a browser.

4. Open /api/books to see the list in JSON.

5. Send POST, PUT and DELETE requests with curl or Postman, including one invalid request, then check the list again.

SETUP AND RUN:

Commands

```
mkdir express-library-api
cd express-library-api
npm init -y
npm install express
node server.js
```

SOURCE CODE:

File Name: server.js

```
const express = require("express");
const app = express();
const PORT = 3000;

app.use(express.json());

// Simple logging middleware
app.use((req, res, next) => {
  console.log(`${req.method} ${req.url}`);
  next();
});

app.get("/", (req, res) => {
  res.send("<h1>Library API</h1><a href='/api/books'>View Books</a>");
});

// In-memory book list
let books = [
  { id: 1, title: "Clean Code", author: "Robert C. Martin", available:
true },
  {
    id: 2,
    title: "You Don't Know JS",
    author: "Kyle Simpson",
    available: true,
  },
];
let nextId = 3;

// Read all books (optional ?available=true filter)
app.get("/api/books", (req, res) => {
  if (req.query.available === "true") {
    return res.json(books.filter((b) => b.available));
  }
  res.json(books);
```

```
});

// Read one book
app.get("/api/books/:id", (req, res) => {
  const book = books.find((b) => b.id === Number(req.params.id));
  if (!book) return res.status(404).json({ error: "Book not found" });
  res.json(book);
});

// Add a book
app.post("/api/books", (req, res) => {
  if (!req.body.title || !req.body.author) {
    return res.status(400).json({ error: "title and author are required"
});
  }
  const book = {
    id: nextId++,
    title: req.body.title,
    author: req.body.author,
    available: true,
  };
  books.push(book);
  res.status(201).json(book);
});

// Issue / return a book
app.put("/api/books/:id", (req, res) => {
  const book = books.find((b) => b.id === Number(req.params.id));
  if (!book) return res.status(404).json({ error: "Book not found" });
  book.available = Boolean(req.body.available);
  res.json(book);
});

// Delete a book
app.delete("/api/books/:id", (req, res) => {
  books = books.filter((b) => b.id !== Number(req.params.id));
  res.json({ message: "Book deleted" });
});

app.listen(PORT, () => {
  console.log(`Server running at http://localhost:${PORT}`);
});
```

OUTPUT:

Server console after node server.js, followed by the requests sent with curl (actual responses). GET returned the two books; POST returned the new book with status 201; a POST without an author returned 400; PUT marked book 2 as issued; the filter ?available=true then showed only the available books; GET for id 99 returned 404; DELETE removed book 1; and the last GET shows the final list. The server console printed one log line for every request.

```
Terminal - server.js

$ node server.js
Server running at http://localhost:3000
GET /api/books
POST /api/books
POST /api/books
PUT /api/books/2
GET /api/books?available=true
GET /api/books/99
DELETE /api/books/1
GET /api/books
```

Figure 6(a): Server console, including the middleware log lines

```
Terminal - curl requests to the Express API (1/2)

$ curl -s http://localhost:3000/api/books
[{"id":1,"title":"Clean Code","author":"Robert C. Martin","available":true},{"id":2,"title":"You Don't
Know JS","author":"Kyle Simpson","available":true}]
$ curl -s -w "\nHTTP %{http_code}\n" -X POST .../api/books -d {"title":"Eloquent
JavaScript","author":"Marijn Haverbeke"}
{"id":3,"title":"Eloquent JavaScript","author":"Marijn Haverbeke","available":true}
HTTP 201
$ curl -s -w "\nHTTP %{http_code}\n" -X POST .../api/books -d {"title":"No Author"}
{"error":"title and author are required"}
HTTP 400
```

Figure 6(b): curl requests, part 1

```
Terminal - curl requests to the Express API (2/2)

$ curl -s -X PUT .../api/books/2 -d {"available":false}
{"id":2,"title":"You Don't Know JS","author":"Kyle Simpson","available":false}
$ curl -s ".../api/books?available=true"
[{"id":1,"title":"Clean Code","author":"Robert C. Martin","available":true},{"id":3,"title":"Eloquent
JavaScript","author":"Marijn Haverbeke","available":true}]
$ curl -s -w "\nHTTP %{http_code}\n" .../api/books/99
{"error":"Book not found"}
HTTP 404
$ curl -s -X DELETE .../api/books/1
{"message":"Book deleted"}
$ curl -s http://localhost:3000/api/books
[{"id":2,"title":"You Don't Know JS","author":"Kyle Simpson","available":false},{"id":3,"title":"Eloquent
JavaScript","author":"Marijn Haverbeke","available":true}]
```

Figure 6(c): curl requests, part 2

RESULT:

A mini application with routing, middleware and REST-style endpoints was developed using Express and Node.js.

VIVA QUESTIONS:

Q1. What is middleware in Express?

Ans: A function that runs between the request and the route handler and can read or modify req and res, e.g. express.json() or the logging function.

Q2. What is the difference between req.params, req.query and req.body?

Ans: req.params holds values from the URL path (like :id); req.query holds the values after ? in the URL; req.body holds the data sent in the request body.

Q3. Why does the logging middleware call next()?

Ans: next() passes control to the next middleware or route handler; without it the request would hang and no response would be sent.

Q4. What is the main limitation of storing books in an array?

Ans: The data lives only in memory and is lost when the server restarts; a database is needed for persistence.

EXPERIMENT 7

CREATE FUNCTIONAL AND CLASS COMPONENTS

USING REACT

[Course Outcome: CO3]

AIM:

To create functional and class components in React and understand props and state.

THEORY:

React builds user interfaces from reusable components. A functional component is a JavaScript function that returns JSX and uses Hooks such as useState for state. A class component extends React.Component, keeps state in this.state and updates it with this.setState(). Props pass data from parent to child components.

IMPORTANT CONCEPTS:

- **JSX:** HTML-like syntax inside JavaScript; {name} inserts the value of any expression, and a style object such as { padding: 15 } is passed in double braces.
- **Props:** read-only inputs passed from parent to child, e.g. <StudentCard name="..." branch="..." year={3} />. One component is reused twice with different props.
- **State:** data that can change and triggers a re-render. useState(false) returns the current value and a setter; the theme toggle flips a Boolean.
- **Updating from the old value:** this.setState(prev => ({ ... })) is used in LikeButton because the new count depends on the previous state.
- **Class property arrow function:** handleClick = () => { ... } keeps this pointing to the component, so no manual binding is needed in the constructor.
- **Hook rules:** call Hooks at the top level of a component, never inside loops or conditions; names start with use.
- **Virtual DOM:** React compares new output with the previous output and updates only what changed in the real DOM.

Feature	Functional component	Class component
Declared as	A function	A class extending Component
State	useState hook	this.state, this.setState()
Lifecycle	useEffect hook	componentDidMount, etc.
Used here for	StudentCard, ThemeToggle	LikeButton

PROCEDURE:

1. Create the project with Vite (npm create vite@latest, select React), then run npm install and npm run dev.
2. Create FunctionalComponent.jsx with StudentCard (props) and ThemeToggle (state).
3. Create ClassComponent.jsx with the LikeButton class.

4. Import them in App.jsx and render them; StudentCard is rendered twice with different props.

5. Click Switch Theme and the Like button and confirm that the two components change independently.

SETUP AND RUN:

Commands

```
npm create vite@latest react-components -- --template react
cd react-components
npm install
npm run dev
```

SOURCE CODE:

File Name: src/FunctionalComponent.jsx

```
import { useState } from "react";

function StudentCard({ name, branch, year }) {
  return (
    <div
      style={{
        border: "1px solid #888",
        borderRadius: 8,
        padding: 10,
        width: 260,
        margin: "10px auto",
      }}
    >
      <h3>{name}</h3>
      <p>
        {branch} - Year {year}
      </p>
    </div>
  );
}

function ThemeToggle() {
  const [dark, setDark] = useState(false);
  const style = {
    background: dark ? "#222" : "#f5f5f5",
    color: dark ? "#fff" : "#222",
    padding: 15,
    margin: "10px auto",
    width: 300,
  };
  return (
    <div style={style}>
      <h3>Functional Component: Theme Toggle</h3>
      <p>Current theme: {dark ? "Dark" : "Light"}</p>
      <button onClick={() => setDark(!dark)}>Switch Theme</button>
```

```
    </div>
  );
}

export { StudentCard, ThemeToggle };
```

File Name: src/ClassComponent.jsx

```
import { Component } from "react";

class LikeButton extends Component {
  constructor(props) {
    super(props);
    this.state = { likes: 0, liked: false };
  }

  handleClick = () => {
    this.setState((prev) => ({
      liked: !prev.liked,
      likes: prev.liked ? prev.likes - 1 : prev.likes + 1,
    }));
  };

  render() {
    return (
      <div>
        <h3>Class Component: Like Button</h3>
        <p>Likes: {this.state.likes}</p>
        <button onClick={this.handleClick}>
          {this.state.liked ? "Unlike" : "Like"}
        </button>
      </div>
    );
  }
}

export default LikeButton;
```

File Name: src/App.jsx

```
import { StudentCard, ThemeToggle } from "./FunctionalComponent";
import LikeButton from "./ClassComponent";

function App() {
  return (
    <div style={{ textAlign: "center", fontFamily: "Arial" }}>
      <h1>React Components Experiment</h1>
      <StudentCard name="Kurmana Sai Sampath" branch="CSE (AI&ML)"
year={3} />
      <StudentCard name="Demo Student" branch="CSE" year={2} />
      <ThemeToggle />
```

```
      <hr />
      <LikeButton />
    </div>
  );
}

export default App;
```

File Name: src/main.jsx (entry file; the Vite template generates it, the unused index.css import removed)

```
import { createRoot } from "react-dom/client";
import App from "./App.jsx";

createRoot(document.getElementById("root")).render(<App />);
```

OUTPUT:

The browser shows the heading, two student cards (the same component with different props), the theme box in the light theme and the like counter at 0. After clicking Switch Theme once and the Like button once, the test run read back “Current theme: Dark” and “Likes: 1” from the page; clicking Unlike returned the count to 0. The project also builds without errors with npx vite build.

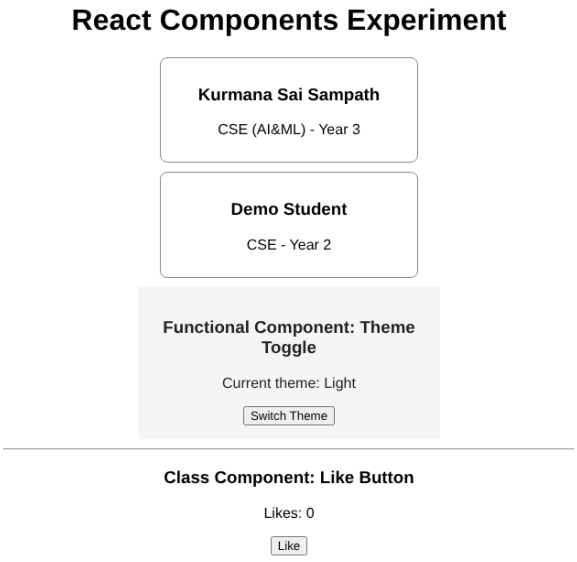


Figure 7(a): Initial render, light theme and 0 likes

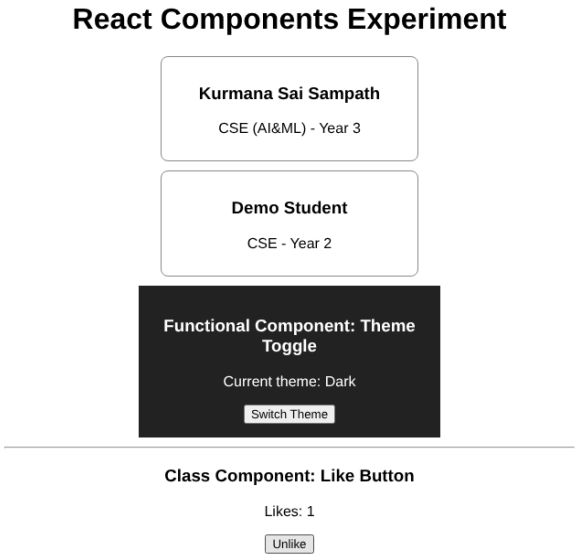


Figure 7(b): After Switch Theme and Like

RESULT:

Functional and class components were created in React, using props and state.

VIVA QUESTIONS:

Q1. What is the difference between props and state?

Ans: Props are read-only inputs from the parent; state is data owned by the component that can change and cause a re-render.

Q2. How is the StudentCard component reused with different data?

Ans: The same function is rendered twice and each use passes different props (name, branch, year), so each card shows its own values.

Q3. Why does LikeButton call setState with a function (prev => ...)?

Ans: The new value depends on the previous state, and React may batch updates, so the function form always receives the latest state.

Q4. Why is setState used instead of changing this.state directly?

Ans: Direct changes do not trigger a re-render; setState updates the state and schedules a re-render.

EXPERIMENT 8

IMPLEMENT HTTP METHODS USING JSON SERVER

[Course Outcome: CO3]

AIM:

To perform GET, POST, PUT, PATCH and DELETE operations on a REST API created with JSON Server.

THEORY:

JSON Server creates a full fake REST API from a JSON file with no coding. HTTP methods map to CRUD operations: GET reads data, POST creates, PUT replaces a resource, PATCH partially updates it and DELETE removes it. Here the API serves a small list of products.

IMPORTANT CONCEPTS:

- **Fake REST API:** JSON Server reads db.json and creates routes for every top-level key; for products it creates /products and /products/:id.
- **PUT vs PATCH:** PUT sends the complete resource and replaces it; PATCH sends only the changed fields, so {"stock": 4} leaves the name and price unchanged.
- **Idempotency:** an idempotent request gives the same result when repeated; sending the same DELETE twice has the same final effect as once.
- **Query features:** /products?price_lte=1000 filters by price, /products?_sort=price&_order=desc sorts, /products?_limit=2 limits the count.
- **Content-Type:** application/json tells the server the body is JSON; without it the body is not parsed.
- **Version:** this record uses JSON Server 0.17.4, which supports the --watch option.

Method	Operation	Idempotent?	Typical status
GET	Read	Yes	200
POST	Create	No	201
PUT	Replace the whole resource	Yes	200
PATCH	Update only given fields	Usually yes	200
DELETE	Remove	Yes	200

PROCEDURE:

1. Install JSON Server with npm install json-server@0.17.4.
2. Create db.json with the products array.
3. Start the server with npx json-server --watch db.json --port 5000.
4. Send the requests in the order GET, POST, PUT, PATCH, a filtered GET and DELETE using requests.sh, curl or Postman.
5. Open db.json after the changes to confirm the file was updated.

SETUP AND START THE SERVER:

Commands

```
mkdir json-server-demo
cd json-server-demo
npm init -y
npm install json-server@0.17.4
npx json-server --watch db.json --port 5000
```

SOURCE CODE:

File Name: db.json

```
{
  "products": [
    {
      "id": 1,
      "name": "Wireless Mouse",
      "category": "Accessories",
      "price": 799,
      "stock": 25
    },
    {
      "id": 2,
      "name": "USB-C Hub",
      "category": "Accessories",
      "price": 1499,
      "stock": 10
    }
  ]
}
```

File Name: requests.sh (run in a second terminal: bash requests.sh)

```
#!/bin/bash
# Experiment 8 - HTTP methods against JSON Server
(http://localhost:5000)
BASE=http://localhost:5000/products
FMT='\nHTTP %{http_code}\n'

echo '$ GET all products'
curl -s -w "$FMT" $BASE

echo '$ POST a new product'
curl -s -w "$FMT" -X POST $BASE -H "Content-Type: application/json" \
  -d '{"name":"Mechanical
Keyboard","category":"Accessories","price":2999,"stock":8}'

echo '$ PUT (replace) product 1'
curl -s -w "$FMT" -X PUT $BASE/1 -H "Content-Type: application/json" \
```

```
  -d '{"name":"Wireless Mouse
Pro","category":"Accessories","price":999,"stock":30}'

echo '$ PATCH (partial update) product 2'
curl -s -w "$FMT" -X PATCH $BASE/2 -H "Content-Type: application/json" \
  -d '{"stock":4}'

echo '$ GET with a filter: price_lte=1000'
curl -s -w "$FMT" "$BASE?price_lte=1000"

echo '$ DELETE product 1'
curl -s -w "$FMT" -X DELETE $BASE/1

echo '$ GET all products (final state)'
curl -s -w "$FMT" $BASE
```

OUTPUT:

The output below is from the actual run. POST returned the new record with the auto-generated id 3 and status 201; PUT replaced product 1 (new name, price and stock); PATCH changed only the stock of product 2; the filter price_lte=1000 returned only the product costing 999 or less; DELETE returned {}; and the final GET shows the updated list.

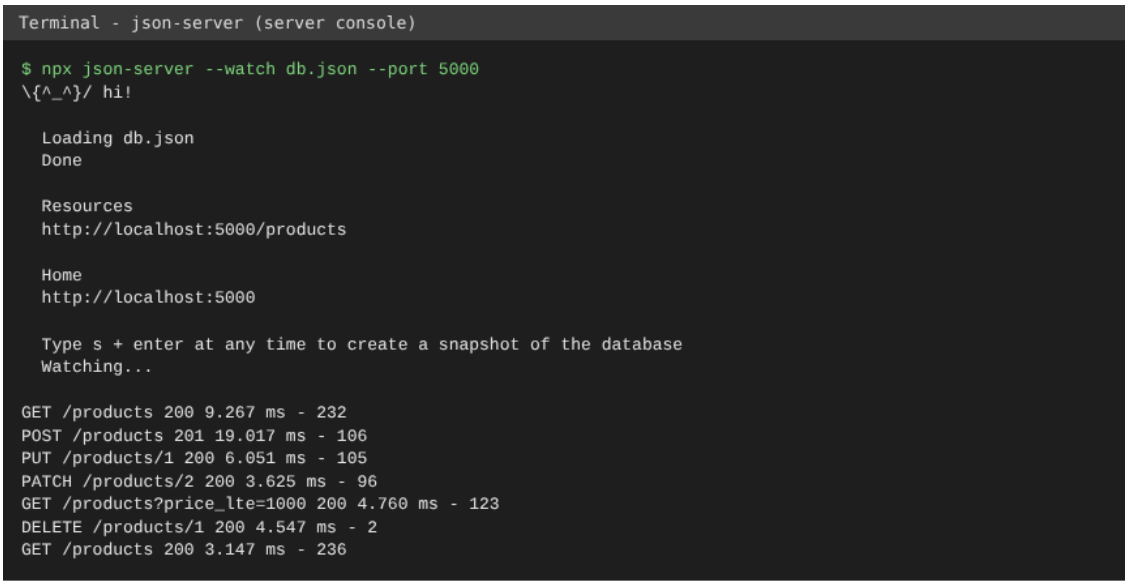


Figure 8(a): JSON Server console

```
Terminal - GET and POST (requests.sh)

$ GET all products
[
  {
    "id": 1,
    "name": "Wireless Mouse",
    "category": "Accessories",
    "price": 799,
    "stock": 25
  },
  {
    "id": 2,
    "name": "USB-C Hub",
    "category": "Accessories",
    "price": 1499,
    "stock": 10
  }
]
HTTP 200
$ POST a new product
{
  "name": "Mechanical Keyboard",
  "category": "Accessories",
  "price": 2999,
  "stock": 8,
  "id": 3
}
HTTP 201
```

Figure 8(b): GET and POST responses

```
Terminal - PUT and PATCH (requests.sh)

$ PUT (replace) product 1
{
  "name": "Wireless Mouse Pro",
  "category": "Accessories",
  "price": 999,
  "stock": 30,
  "id": 1
}
HTTP 200
$ PATCH (partial update) product 2
{
  "id": 2,
  "name": "USB-C Hub",
  "category": "Accessories",
  "price": 1499,
  "stock": 4
}
HTTP 200
```

Figure 8(c): PUT and PATCH responses

```
Terminal - filter, DELETE and final GET (requests.sh)

$ GET with a filter: price_lte=1000
[
  {
    "name": "Wireless Mouse Pro",
    "category": "Accessories",
    "price": 999,
    "stock": 30,
    "id": 1
  }
]
HTTP 200
$ DELETE product 1
{}
HTTP 200
$ GET all products (final state)
[
  {
    "id": 2,
    "name": "USB-C Hub",
    "category": "Accessories",
    "price": 1499,
    "stock": 4
  },
  {
    "name": "Mechanical Keyboard",
    "category": "Accessories",
    "price": 2999,
    "stock": 8,
    "id": 3
  }
]
HTTP 200
```

Figure 8(d): Filtered GET, DELETE and final GET responses

RESULT:

All five HTTP methods were implemented and verified against a JSON Server REST API.

VIVA QUESTIONS:

Q1. What is the difference between PUT and PATCH?

Ans: PUT replaces the entire resource; PATCH changes only the fields that are sent.

Q2. What does idempotent mean? Which methods are idempotent?

Ans: Repeating the request has the same effect as sending it once. GET, PUT and DELETE are idempotent; POST is not.

Q3. How can the products costing at most 1000 be fetched from JSON Server?

Ans: By sending GET /products?price_lte=1000; the suffix _lte means “less than or equal to”.

Q4. Why is the Content-Type header needed when sending JSON?

Ans: It tells the server to parse the body as JSON; otherwise req.body is empty or unparsed.

EXPERIMENT 9

DEVELOP CRUD OPERATIONS USING MONGODB

(MONGOOSE AND MONGOSH)

[Course Outcome: CO4]

AIM:

To perform Create, Read, Update and Delete operations on a MongoDB database, first from the MongoDB shell (mongosh) and then from a Node.js application using Mongoose.

THEORY:

MongoDB is a NoSQL document database that stores data as BSON documents inside collections. A database holds collections and a collection holds documents. Mongoose is an Object Data Modelling (ODM) library for Node.js that defines schemas and provides an easy API for MongoDB operations. CRUD corresponds to insertOne()/find(), updateOne() and deleteOne() in the shell, and to create(), find(), updateOne() and deleteOne() in Mongoose. This experiment uses a books collection in a database named libraryDB.

IMPORTANT CONCEPTS:

- **Hierarchy:** server → databases → collections → documents (key-value pairs stored as BSON).
- **_id:** every document gets a unique ObjectId when created, used to find, update and delete it.
- **Mongoose:** a schema defines the shape, validation and defaults (required: true rejects a book with no title, default: true gives inStock its value); a model such as Book runs queries and maps to the collection books.
- **Range queries and sorting:** { price: { $gte: 400, $lte: 700 } } selects a range and .sort({ price: 1 }) orders it from low to high.
- **Connection:** mongoose.connect() returns a promise; mongoose.disconnect() closes the connection when the script ends.
- **Shell vs code:** the shell is handy for checking data; Mongoose is used inside applications.

Operator	Meaning	Example
$gt, $gte	Greater than (or equal)	{ price: { $gt: 500 } }
$lt, $lte	Less than (or equal)	{ price: { $lt: 500 } }
$set	Sets a field in an update	{ $set: { price: 599 } }
$inc	Adds a number to a field	{ $inc: { price: 20 } }

PROCEDURE:

1. Start the MongoDB service, then open a terminal and run mongosh.

2. Switch to libraryDB with use libraryDB and insert books with insertOne and insertMany.

3. Run the READ queries with filters, a range and sorting, then update and delete.

4. Create the Node.js project, install Mongoose and write crud.js.

5. Run node crud.js on an empty collection and compare the output with the expected output.

SETUP AND COMMANDS:

Commands

```
# Part A - mongosh
mongod --dbpath "C:\data\db"   # only if MongoDB is not already running
as a service
mongosh                          # connect to the local server
show dbs                         # list databases
use libraryDB                    # switch to (or create) libraryDB
show collections

# Part B - Mongoose
mkdir mongo-crud && cd mongo-crud
npm init -y
npm install mongoose
node crud.js
```

SOURCE CODE:

File Name: mongosh_commands.js (Part A - mongosh)

```
// Run inside mongosh (paste line by line, or: mongosh --file
mongosh_commands.js)
use("libraryDB");

// CREATE
db.books.insertOne({
  title: "Clean Code",
  author: "Robert C. Martin",
  price: 650,
});
db.books.insertMany([
  { title: "Eloquent JavaScript", author: "Marijn Haverbeke", price: 480
},
  { title: "The Pragmatic Programmer", author: "Andrew Hunt", price: 720
},
]);

// READ
db.books.find(); // all documents
db.books.find({ author: "Marijn Haverbeke" }); // filter by field
db.books.find({ price: { $gt: 500 } }); // price greater than 500
db.books.find({ price: { $gte: 400, $lte: 700 } }).sort({ price: 1 });
// range, sorted
db.books.findOne({ title: "Clean Code" }); // first matching document

// UPDATE
db.books.updateOne({ title: "Clean Code" }, { $set: { price: 599 } });
db.books.updateMany({}, { $set: { inStock: true } });
db.books.updateOne({ title: "Eloquent JavaScript" }, { $inc: { price: 20
} });
```

```
// DELETE
db.books.deleteOne({ title: "Eloquent JavaScript" });
db.books.deleteMany({ price: { $lt: 100 } });
```

File Name: crud.js (Part B - Mongoose)

```
const mongoose = require("mongoose");

mongoose
  .connect("mongodb://127.0.0.1:27017/libraryDB")
  .then(() => console.log("Connected to MongoDB"))
  .catch((err) => console.error("Connection error:", err));

// Define schema and model
const bookSchema = new mongoose.Schema({
  title: { type: String, required: true },
  author: String,
  price: Number,
  inStock: { type: Boolean, default: true },
});
const Book = mongoose.model("Book", bookSchema);

async function runCrud() {
  // CREATE
  const b1 = await Book.create({
    title: "Clean Code",
    author: "Robert C. Martin",
    price: 650,
  });
  await Book.create({
    title: "Eloquent JavaScript",
    author: "Marijn Haverbeke",
    price: 480,
  });
  console.log("Created:", b1);

  // READ
  const all = await Book.find();
  console.log("Total books:", all.length);
  const cheap = await Book.find({ price: { $lt: 500 } });
  console.log(
    "Books under Rs. 500:",
    cheap.map((b) => b.title),
  );

  // UPDATE
  await Book.updateOne({ title: "Clean Code" }, { price: 599 });
  const updated = await Book.findOne({ title: "Clean Code" });
  console.log("New price of Clean Code:", updated.price);
```

```
  // DELETE
  await Book.deleteOne({ title: "Eloquent JavaScript" });
  console.log("Books left:", await Book.countDocuments());

  await mongoose.disconnect();
}

runCrud();
```

OUTPUT:

Part A - mongosh (expected output) - UNVERIFIED: expected output, not executed

Expected Output

```
test> use libraryDB
switched to db libraryDB
libraryDB> db.books.insertOne({ title: "Clean Code", author: "Robert C.
Martin", price: 650 })
{ acknowledged: true, insertedId: ObjectId('...') }
libraryDB> db.books.find({ price: { $gt: 500 } })
[
  { _id: ObjectId('...'), title: 'Clean Code', author: 'Robert C.
Martin', price: 650 },
  { _id: ObjectId('...'), title: 'The Pragmatic Programmer', author:
'Andrew Hunt', price: 720 }
]
libraryDB> db.books.updateOne({ title: "Clean Code" }, { $set: { price:
599 } })
{ acknowledged: true, insertedId: null, matchedCount: 1, modifiedCount:
1, upsertedCount: 0 }
libraryDB> db.books.deleteOne({ title: "Eloquent JavaScript" })
{ acknowledged: true, deletedCount: 1 }
```

Part B - node crud.js (expected output, on an empty collection) - UNVERIFIED: expected output, not executed

Expected Output

```
Connected to MongoDB
Created: { title: 'Clean Code', author: 'Robert C. Martin', price: 650,
inStock: true, _id: new ObjectId('...'), __v: 0 }
Total books: 2
Books under Rs. 500: [ 'Eloquent JavaScript' ]
New price of Clean Code: 599
Books left: 1
```

Verification status: MongoDB could not be downloaded or started in the environment where this record was prepared (the download was blocked), so these commands were not executed and no screenshot of them is included. The output above is the expected output, marked unverified. What was checked: crud.js passes node --check, and the schema was exercised offline with a small script

(schema_check.js): a book without a title is rejected with “Path `title` is required.”, the model maps to the collection books, and inStock defaults to true. Run the commands against a local MongoDB and paste a screenshot of your own terminal over this section.

RESULT:

Create, Read, Update and Delete operations were written for a MongoDB collection using the mongosh shell and using Mongoose; execution against a live MongoDB is pending (see verification status).

VIVA QUESTIONS:

Q1. What is the difference between a collection and a document?

Ans: A document is one record (a set of key-value pairs); a collection is a group of documents, similar to a table.

Q2. What is the purpose of _id?

Ans: It is the unique primary key of a document, generated automatically as an ObjectId.

Q3. What is the difference between $set and $inc in an update?

Ans: $set replaces the value of a field with the given value; $inc adds the given number to the current value of the field.

Q4. What is a Mongoose schema and why is it useful?

Ans: It defines the fields, types, defaults and validation rules of documents, adding structure to schema-less MongoDB.

EXPERIMENT 10

DEVELOP MAIN PROJECT USING EXPRESS,

REACT AND MONGODB

[Course Outcome: CO4]

AIM:

To develop a full-stack application (a Student Management System) using Express for the backend, React for the frontend and MongoDB for the database.

THEORY:

A full-stack MERN-style application separates concerns into three layers. MongoDB stores data, Express with Node.js exposes REST APIs, and React consumes those APIs to render and manage the user interface. The frontend communicates with the backend over HTTP using fetch, and CORS must be enabled so that the React app on one port can call the server on another.

IMPORTANT CONCEPTS:

- **MERN stack:** MongoDB (database), Express (server), React (user interface) and Node.js (runtime), connected only through well-defined interfaces.
- **CORS:** browsers block a page from calling a server on a different origin (a different port counts). cors() adds the headers that allow it.
- **Schema validation:** the Student schema requires a name and limits cgpa to the range 0–10; runValidators: true makes the same rules apply to updates.
- **Async code:** database calls use async/await; Express 5 passes a rejected promise to its error handler, which answers with status 500.
- **Frontend state:** useState holds the student list, the form values, the id being edited and the search text; useEffect(() => ..., []) loads the list once when the page opens.
- **Derived data:** the table shows students.filter(...) of the search text, so searching needs no extra request to the server.
- **Keys:** key={s._id} lets React track each row when the list changes.
- **CRUD mapping:** Create = POST, Read = GET, Update = PUT (Edit button), Delete = DELETE.

Architecture

Architecture

```
React (localhost:5173)
    |   fetch() with JSON over HTTP
    v
Express API (localhost:5000)
    |   Mongoose queries
    v
MongoDB (localhost:27017)

Responses travel back along the same path as JSON.
```

PROCEDURE:

1. Start MongoDB, then run node server.js in the server folder and confirm “MongoDB connected”.

2. Create the Vite React project in client, put the code below in src/App.jsx, and run npm run dev.

3. Open http://localhost:5173 and add a student with a name, branch and CGPA.

4. Confirm that the new row appears in the table and in MongoDB (db.students.find() in mongosh).

5. Type part of a name in the search box, then click Edit, change a value and press Update Student; finally click Delete on a row and confirm it disappears from the table and the database.

PROJECT STRUCTURE:

Folder Structure

```
student-management/
|-- server/
|   |-- server.js
|   `-- package.json
`-- client/            (created with Vite + React)
    |-- index.html
    |-- vite.config.js
    `-- src/
        |-- main.jsx
        `-- App.jsx
```

SETUP AND RUN:

Commands

```
mkdir student-management && cd student-management
mkdir server && cd server
npm init -y
npm install express mongoose cors
node server.js          # terminal 1
cd ..
npm create vite@latest client -- --template react
cd client
npm install
npm run dev             # terminal 2
```

SOURCE CODE:

File Name: server/server.js

```
const express = require("express");
const mongoose = require("mongoose");
const cors = require("cors");

const app = express();
app.use(cors());
app.use(express.json());
```

```
mongoose
  .connect("mongodb://127.0.0.1:27017/studentDB")
  .then(() => console.log("MongoDB connected"))
  .catch((err) => console.error(err));

const Student = mongoose.model(
  "Student",
  new mongoose.Schema({
    name: { type: String, required: true },
    branch: String,
    cgpa: { type: Number, min: 0, max: 10 },
  }),
);

// Read all students
app.get("/api/students", async (req, res) => {
  const students = await Student.find();
  res.json(students);
});

// Create a student
app.post("/api/students", async (req, res) => {
  const student = await Student.create(req.body);
  res.status(201).json(student);
});

// Update a student
app.put("/api/students/:id", async (req, res) => {
  const student = await Student.findByIdAndUpdate(req.params.id,
req.body, {
    new: true,
    runValidators: true,
  });
  res.json(student);
});

// Delete a student
app.delete("/api/students/:id", async (req, res) => {
  await Student.findByIdAndDelete(req.params.id);
  res.json({ message: "Deleted" });
});

app.listen(5000, () => console.log("Server running on
http://localhost:5000"));
```

File Name: client/src/App.jsx

```
import { useState, useEffect } from "react";

const API = "http://localhost:5000/api/students";
```

```
const emptyForm = { name: "", branch: "", cgpa: "" };

function App() {
  const [students, setStudents] = useState([]);
  const [form, setForm] = useState(emptyForm);
  const [editId, setEditId] = useState(null);
  const [search, setSearch] = useState("");

  const loadStudents = async () => {
    const res = await fetch(API);
    setStudents(await res.json());
  };

  useEffect(() => {
    loadStudents();
  }, []);

  // Create (POST) or update (PUT) depending on editId
  const saveStudent = async (e) => {
    e.preventDefault();
    const body = JSON.stringify({ ...form, cgpa: Number(form.cgpa) });
    await fetch(editId ? `${API}/${editId}` : API, {
      method: editId ? "PUT" : "POST",
      headers: { "Content-Type": "application/json" },
      body,
    });
    setForm(emptyForm);
    setEditId(null);
    loadStudents();
  };

  const startEdit = (s) => {
    setEditId(s._id);
    setForm({ name: s.name, branch: s.branch, cgpa: s.cgpa });
  };

  const deleteStudent = async (id) => {
    await fetch(`${API}/${id}`, { method: "DELETE" });
    loadStudents();
  };

  const visible = students.filter((s) =>
    s.name.toLowerCase().includes(search.toLowerCase()),
  );

  return (
    <div style={{ fontFamily: "Arial", padding: 20 }}>
      <h1>Student Management System</h1>
      <form onSubmit={saveStudent}>
        <input
          placeholder="Name"
          value={form.name}
```

```
          required
          onChange={(e) => setForm({ ...form, name: e.target.value })}
        />
        <input
          placeholder="Branch"
          value={form.branch}
          onChange={(e) => setForm({ ...form, branch: e.target.value })}
        />
        <input
          placeholder="CGPA"
          type="number"
          step="0.01"
          min="0"
          max="10"
          value={form.cgpa}
          onChange={(e) => setForm({ ...form, cgpa: e.target.value })}
        />
        <button type="submit">
          {editId ? "Update Student" : "Add Student"}
        </button>
      </form>
      <input
        style={{ marginTop: 15 }}
        placeholder="Search by name"
        value={search}
        onChange={(e) => setSearch(e.target.value)}
      />
      <table border="1" cellPadding="8" style={{ marginTop: 15 }}>
        <thead>
          <tr>
            <th>Name</th>
            <th>Branch</th>
            <th>CGPA</th>
            <th>Action</th>
          </tr>
        </thead>
        <tbody>
          {visible.map((s) => (
            <tr key={s._id}>
              <td>{s.name}</td>
              <td>{s.branch}</td>
              <td>{s.cgpa}</td>
              <td>
                <button onClick={() => startEdit(s)}>Edit</button>
                <button onClick={() =>
deleteStudent(s._id)}>Delete</button>
              </td>
            </tr>
          ))}
        </tbody>
      </table>
    </div>
```

```
  );
}

export default App;
```

File Name: client/src/main.jsx (generated by the Vite template; unused CSS import removed)

```
import { createRoot } from "react-dom/client";
import App from "./App.jsx";

createRoot(document.getElementById("root")).render(<App />);
```

OUTPUT:

Backend terminal (expected output) - UNVERIFIED: expected output, not executed

Expected Output

```
$ node server.js
MongoDB connected
Server running on http://localhost:5000
```

Expected browser behaviour (unverified): the React page at http://localhost:5173 shows a form for adding students, a search box and a table of all students stored in MongoDB. Adding a student inserts it into the table immediately, typing in the search box narrows the rows, Edit loads a row into the form for an update, and Delete removes the row from both the table and the database.

Verification status: MongoDB could not be started in the environment where this record was prepared, so the full application was not run end to end and no screenshots of it are included. What was checked: server.js passes node --check, and the React client compiles with npx vite build without errors. Run the steps above on a machine with MongoDB and add your own screenshots of the terminal and the browser page.

RESULT:

A full-stack Student Management System was developed using Express, React and MongoDB, with create, read, update, delete and search functionality; end-to-end execution is pending (see verification status).

VIVA QUESTIONS:

Q1. What do the letters M, E, R and N stand for in MERN?
Ans: MongoDB, Express, React and Node.js.

Q2. Why is CORS needed when React and Express run on different ports?
Ans: Browsers treat different ports as different origins and block the requests unless the server sends CORS headers.

Q3. How does the search box filter the table without calling the server?
Ans: The component keeps the search text in state and renders students.filter(...) on each render, so only matching rows are shown and the data is already in memory.

Q4. Why does the useEffect in this application have an empty dependency array?

Ans: So the students are fetched only once, when the component first mounts, instead of after every render.

FINAL CONCLUSION

The Full-Stack Development Laboratory gave a complete, hands-on journey through the layers of a modern web application. Beginning with the fundamentals of HTML, CSS and JavaScript in Experiments 1 to 4 (CO1), I learned how to structure a page with semantic elements, style it responsively with CSS3 using Flexbox and Grid, and make it interactive by handling events and updating the DOM without reloading the page. The marks-analysis program strengthened my core JavaScript logic with arrays, sorting, recursion and higher-order functions, which supports every later experiment.

In Experiments 5 and 6 (CO2), I moved to the server side. Building custom and built-in modules in Node.js showed me how code is organised into reusable units, and the Express library API taught routing, middleware, query strings, JSON request handling and REST-style endpoints with correct HTTP status codes such as 201, 400 and 404. Experiment 7 (CO3) introduced React, where functional and class components, props, state and Hooks showed how a user interface can be built from small reusable parts that re-render efficiently.

Experiment 8 (CO3) tied the layers together by exercising the GET, POST, PUT, PATCH and DELETE methods, and filtered queries, against a REST API created with JSON Server, and clarified the difference between replacing and partially updating a resource.

Experiments 9 and 10 (CO4) covered the data layer and the complete application. MongoDB CRUD operations were written both in the mongosh shell and in Node.js using Mongoose schemas, and the Student Management System combined Express, React and MongoDB into one working MERN-style design, with CORS enabled so the React client can call the API. Experiments 1 to 8 were executed and their outputs and screenshots are genuine. Experiments 9 and 10 need a running MongoDB instance; their code was syntax-checked, the Mongoose schema rules were tested offline and the React client was built, and their outputs are marked as expected and unverified until they are run against a local database.

Through these ten experiments I learned that a full-stack application is a set of well-defined layers that communicate through clear interfaces: the browser renders the interface, the server exposes the logic through an API, and the database stores the data permanently. I also gained practical habits that matter beyond the laboratory, such as validating user input, keeping behaviour separate from markup, testing APIs with tools like curl, and reading error messages carefully while debugging.

In conclusion, this laboratory achieved all four course outcomes. It gave me a solid foundation in front-end design, server-side programming, component-based user interfaces and database-driven applications, and it prepared me to design, build and test complete web applications. I am grateful to my faculty guide for the guidance that made this possible, and I will carry these skills into my future coursework and projects.